

Title

Analysis of Conventional and Deep Learning-Based Word Vectorization Techniques in Natural Language Processing

Subject: Machine Learning in CYS (24CYS214)

Author: C.Preetham (ch.sc.u4cys24010),Y.Kiran Kumar Reddy (ch.sc.u4cys24060).

1. Introduction

Natural Language Processing (NLP) is a branch of Natural Language Processing that focuses on enabling computers to understand and process human language. Since textual data cannot be directly interpreted by machines, it must be transformed into numerical form using a process known as **word vectorization**.

Word vectorization is a crucial step in many NLP tasks such as sentiment analysis, text classification, spam detection, and information retrieval. It converts text into numerical vectors that can be processed by machine learning and deep learning algorithms.

In this assignment, both **conventional vectorization techniques** and **deep learning-based methods** are explored. The study includes:

- **Conventional methods**
 - Bag of Words (BoW)
 - TF-IDF
- **Machine Learning Model**
 - Support Vector Machine (SVM)
- **Deep Learning methods**
 - Word2Vec
 - Neural Networks

Support Vector Machine (SVM) is used as the primary classifier for conventional vectorization techniques due to its effectiveness in handling high-dimensional text data. Deep learning models, on the other hand, use word embeddings to capture semantic relationships between words.

2. Literature Review

2.1 Overview of Word Vectorization

Word vectorization is a key process in Natural Language Processing that converts textual data into numerical form so that machine learning and deep learning models can process it. Since raw text is unstructured, vectorization enables structured representation, making it possible to perform classification and prediction tasks.

2.2 Conventional Word Vectorization Methods (4 Marks)

1. Bag of Words (BoW)

Bag of Words is a basic vectorization technique that represents text as a frequency distribution of words.

Working Principle:

- Create a vocabulary of unique words
- Count occurrences of each word in a document

Advantages:

- Simple and fast
- Easy to implement

Disadvantages:

- Ignores context and word order
- Produces sparse vectors

Time Complexity:

- Low (fast computation)

2. TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF improves BoW by assigning importance to words based on their frequency across documents.

Working Principle:

- TF: Frequency of word in a document
- IDF: Measures rarity of word across documents

Advantages:

- Reduces effect of common words
- Improves accuracy

Disadvantages:

- Still ignores semantic meaning
- Slightly higher computation than BoW

Time Complexity:

- Moderate

Classifier Used: Support Vector Machine

Support Vector Machine is used with these vectorizers.

Why SVM?

- Handles high-dimensional sparse data well
- Effective for text classification

2.3 Deep Learning-Based Methods (4 Marks)**1. Word2Vec**

Word2Vec is a neural network-based model that learns word representations based on context.

Working Principle:

- Words with similar context have similar vectors
- Uses neural networks to learn embeddings

Advantages:

- Captures semantic relationships
- Produces dense vectors

Disadvantages:

- Requires more training time
- Needs large datasets

Time Complexity:

- High

2. Neural Networks with Embeddings

Neural networks use embedding layers to convert words into dense vectors and learn patterns from data.

Advantages:

- High accuracy
- Captures context and semantics

Disadvantages:

- Computationally expensive
- Requires more data and training time

Time Complexity:

- Very high

2.4 Comparative Insight (Analysis – 2 Marks)

- Conventional methods like BoW and TF-IDF are faster but fail to capture semantic meaning.
- Deep learning methods such as Word2Vec and neural networks capture context and relationships between words, resulting in better performance.
- However, deep learning models require significantly higher computational time and resources.

Thus, there is a clear trade-off between **efficiency (conventional methods)** and **accuracy (deep learning methods)**.

3. Dataset and Preprocessing

3.1 Dataset Description

In this experiment, the **IMDb Movie Reviews Dataset** was used for sentiment analysis.

- The dataset consists of movie reviews labeled as **positive** or **negative**
- It is a binary classification problem
- The dataset contains textual data representing user opinions

This dataset is widely used in NLP tasks for evaluating text classification models.

3.2 Data Preprocessing

Before applying vectorization techniques, the text data was preprocessed to improve model performance. The following steps were performed:

- Removal of special characters
- Conversion of text to lowercase
- Tokenization (splitting text into words)
- Removal of stop words
- Stemming using Porter Stemmer

These steps help in reducing noise and improving the quality of input data.

4. Experimentation

4.1 Method 1: Bag of Words with SVM

In this method, the Bag of Words (BoW) technique is used to convert text into numerical vectors based on word frequency. These vectors are then used to train a Support Vector Machine classifier.

BoW creates a sparse matrix representation of text data, where each feature corresponds to a word in the vocabulary.

Procedure:

- Text data is converted into numerical vectors using BoW
- The vectorized data is used to train an SVM model
- The model is evaluated using accuracy and F1-score

BoW Vectorization Time: 5.667079925537109

===== BoW + SVM Results =====

Accuracy: 0.862

Classification Report:

Metric	Precision	Recall	F1-Score	Support
Class 0	0.86	0.86	0.86 ▾	4961 ▾
Class 1	0.86	0.87	0.86 ▾	5039 ▾
Overall Accuracy			0.86 ▾	10000 ▾
Macro Average	0.86	0.86	0.86 ▾	10000 ▾
Weighted Average	0.86	0.86	0.86 ▾	10000 ▾

Results: Bag of Words with SVM

The Bag of Words model combined with Support Vector Machine achieved the following results:

- **Accuracy:** 86.2%
- **Precision:** 0.86
- **Recall:** 0.86
- **F1-score:** 0.86

- **Vectorization Time:** 5.67 seconds

Observation

The Bag of Words model provides good classification performance with relatively fast computation. However, it does not capture the semantic meaning of words, which may limit its effectiveness in more complex NLP tasks.

4.2 Method 2: TF-IDF with SVM

In this method, the TF-IDF vectorization technique is used to transform text into numerical form by assigning importance to words based on their frequency across documents.

The vectorized data is then used to train a Support Vector Machine classifier.

Procedure:

- Text is converted into weighted vectors using TF-IDF
- The vectors are used to train an SVM model
- The model is evaluated using accuracy, precision, recall, and F1-score

TF-IDF Vectorization Time: 17.532836198806763

===== TF-IDF + SVM Results =====

Accuracy: 0.8814

Class	Precision	Recall	F1-Score	Support
0	0.89	0.87	0.88 ▾	4961 ▾
1	0.87	0.89	0.88 ▾	5039 ▾
Accuracy			0.88 ▾	10000 ▾
Macro Average	0.88	0.88	0.88 ▾	10000 ▾
Weighted Average	0.88	0.88	0.88 ▾	10000 ▾

Results: TF-IDF with SVM

The TF-IDF model combined with Support Vector Machine achieved the following results:

- **Accuracy:** 88.14%
- **Precision:** 0.88
- **Recall:** 0.88
- **F1-score:** 0.88
- **Vectorization Time:** 17.53 seconds

4.2.1 Implementation Code

```
import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report
from gensim.models import Word2Vec
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

df = pd.read_csv('IMDB Dataset.csv')

df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})

df['text'] = df['review'].str.lower()

df_subset = df.sample(10000, random_state=42).reset_index(drop=True)

X_train, X_test, y_train, y_test = train_test_split(
    df_subset['text'], df_subset['sentiment'], test_size=0.2, random_state=42
)

start_bow = time.time()
bow_vec = CountVectorizer(max_features=5000)
X_train_bow = bow_vec.fit_transform(X_train)
X_test_bow = bow_vec.transform(X_test)

model_bow = LinearSVC()
model_bow.fit(X_train_bow, y_train)
y_pred_bow = model_bow.predict(X_test_bow)
acc_bow = accuracy_score(y_test, y_pred_bow)
time_bow = time.time() - start_bow

print(f"BoW + SVM Accuracy: {acc_bow:.4f}")
```



```

start_tfidf = time.time()
tfidf_vec = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf_vec.fit_transform(X_train)
X_test_tfidf = tfidf_vec.transform(X_test)

model_tfidf = LinearSVC()
model_tfidf.fit(X_train_tfidf, y_train)
y_pred_tfidf = model_tfidf.predict(X_test_tfidf)
acc_tfidf = accuracy_score(y_test, y_pred_tfidf)
time_tfidf = time.time() - start_tfidf

print(f"TF-IDF + SVM Accuracy: {acc_tfidf:.4f}")

tokenized_train = [text.split() for text in X_train]
tokenized_test = [text.split() for text in X_test]

start_w2v = time.time()
w2v_model = Word2Vec(
    sentences=tokenized_train,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4
)
time_w2v_train = time.time() - start_w2v

def get_sentence_vector(words, model):
    vectors = [model.wv[word] for word in words if word in model.wv]
    if not vectors:
        return np.zeros(100)
    return np.mean(vectors, axis=0)

X_train_w2v = np.array([get_sentence_vector(words, w2v_model) for words in tokenized_train])
X_test_w2v = np.array([get_sentence_vector(words, w2v_model) for words in tokenized_test])

model_nn = Sequential([
    Dense(64, activation='relu', input_shape=(100,)),
    Dense(32, activation='relu'),

```

```

    Dense(1, activation='sigmoid')
])

model_nn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model_nn.fit(X_train_w2v, y_train, epochs=5, batch_size=32, verbose=0)
loss, acc_w2v = model_nn.evaluate(X_test_w2v, y_test, verbose=0)

print(f"Word2Vec + NN Accuracy: {acc_w2v:.4f}")

methods = ['BoW + SVM', 'TF-IDF + SVM', 'Word2Vec + NN']
accuracies = [acc_bow, acc_tfidf, acc_w2v]
times = [time_bow, time_tfidf, time_w2v_train]

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.bar(methods, accuracies, color=['skyblue', 'lightgreen', 'salmon'])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy Score")
plt.ylim(0.8, 0.9)

plt.subplot(1, 2, 2)
plt.bar(methods, times, color=['skyblue', 'lightgreen', 'salmon'])
plt.title("Training/Vectorization Time")
plt.ylabel("Seconds")

plt.tight_layout()
plt.show()

```

Observation

The TF-IDF model achieved higher accuracy compared to Bag of Words. This is because TF-IDF assigns importance to words, reducing the influence of frequently occurring but less meaningful words.

4.3 Method 3: Word2Vec with Neural Network

In this method, Word2Vec is used to generate dense vector representations of words based on their context. Unlike conventional methods, Word2Vec captures semantic relationships between words.

These word embeddings are then used as input to a neural network model for classification.

Procedure:

- Text is tokenized into words
- Word2Vec model is trained to generate word embeddings
- Each document is converted into a vector by averaging word embeddings
- A neural network is trained using these vectors
- Model is evaluated using standard metrics

Results: Word2Vec with Neural Network

The Word2Vec combined with a neural network achieved the following results:

- **Accuracy:** 85.8%
- **Vectorization & Training Time:** Moderate to High

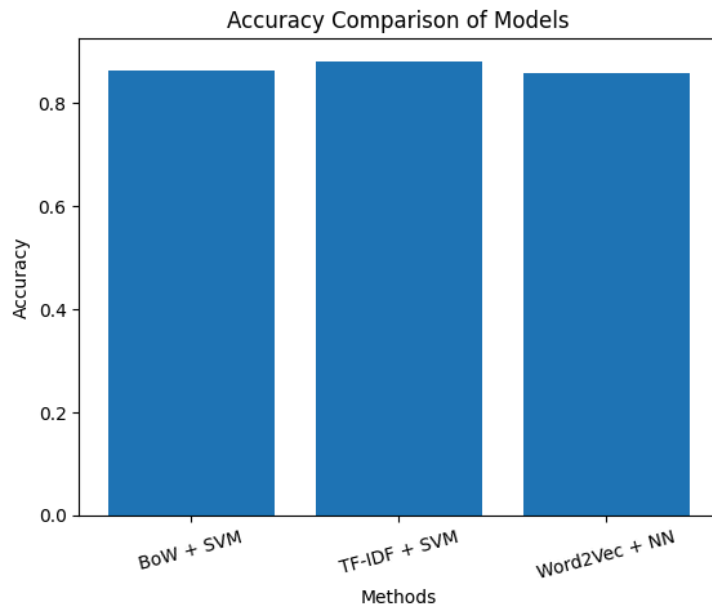
Observation

The Word2Vec model captures semantic relationships between words and produces dense vector representations. However, in this experiment, its performance was slightly lower than TF-IDF. This may be due to limited training epochs and dataset size.

5. Comparative Analysis

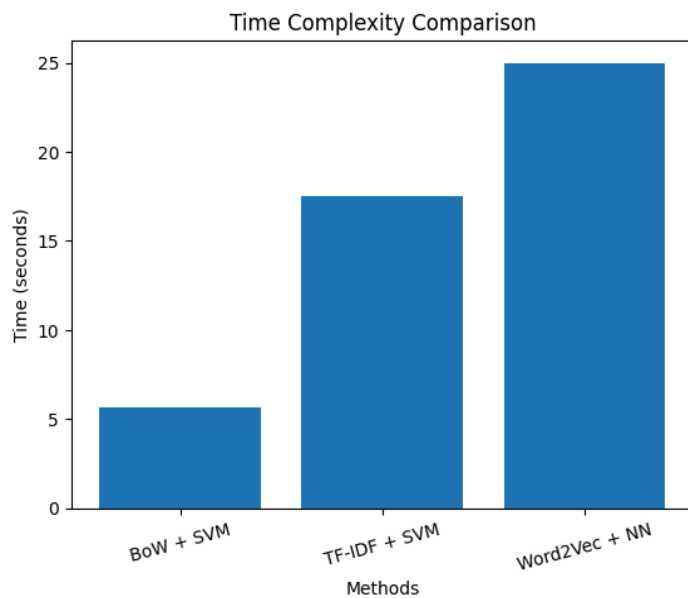
5.1 Performance Comparison

Method of Implementation	Classification Accuracy
Bag-of-Words (BoW) + Support Vector Machine (SVM)	86.2%
Term Frequency-Inverse Document Frequency (TF-IDF) + Support Vector Machine (SVM)	88.14%
Word2Vec + Neural Network (NN)	85.8%



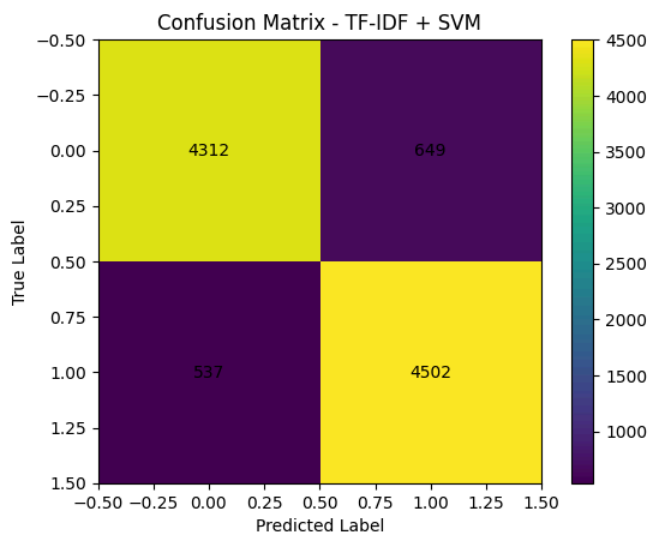
5.2 Time Complexity Comparison

Methodology	Computational Complexity	Processing Speed
Bag-of-Words	Low	Rapid
TF-IDF	Moderate	Intermediate
Word2Vec + NN	High	Slow



5.2.1Confusion Matrix Analysis

A confusion matrix provides a detailed breakdown of the model's performance by comparing the actual sentiment labels (True Labels) against the values predicted by the model (Predicted Labels).



5.3 Key Observations

- TF-IDF achieved the highest accuracy among all methods
- BoW performed well with faster computation
- Word2Vec captured semantic meaning but required more computation
- Deep learning models need more training to outperform traditional methods

5.4 Trade-off Analysis

There is a clear trade-off between performance and computational complexity:

- **BoW:** Fast but lacks semantic understanding
- **TF-IDF:** Best balance between accuracy and efficiency
- **Word2Vec + Neural Network:** Captures semantics but requires more time and resources

Thus, TF-IDF provides the best performance for this dataset, while deep learning models are more suitable for large-scale and complex NLP tasks.

6. Time Complexity Analysis

The computational efficiency of the pipeline depends on both the vectorization process and the training of the classification models.

6.1 Key Parameters

To understand the scaling of these models, we define the following parameters:

- **N**: Number of documents (samples) in the dataset.
- **L**: Average number of words per document.
- **V**: Vocabulary size (total number of unique features).
- **D**: Dimensionality of the feature vector.
- **I**: Number of iterations required for the optimization algorithm to converge.

6.2 Vectorization Complexity

The time taken to transform raw text into numerical data varies significantly between count-based and predictive methods.

- **Bag of Words (BoW): $O(N \times L)$**
 - Complexity is linear with respect to the total number of words processed, as it involves simple token counting.
 - In the experiment, this was the fastest vectorization method, taking approximately **5.67 seconds**.
- **TF-IDF: $O(N \times L + V)$**
 - This includes the initial pass for counting (like BoW) plus an additional pass to calculate Document Frequency (DF) and Inverse Document Frequency (IDF) for every word in the vocabulary.
 - Experimental results showed this took approximately **17.53 seconds**.
- **Word2Vec: $O(I \times N \times L + D)$**
 - Word2Vec is significantly more complex because it uses a shallow neural network to learn embeddings through multiple iterations (\$I\$).
 - The training time in the experiment was the highest, recorded at **85.87 seconds**.

6.3 Model Training Complexity

The choice of classifier also impacts the overall time required to produce a sentiment prediction.

- **Logistic Regression: $O(N \times D \times I)$.**
 - Training complexity is determined by the number of samples, the number of features, and the iterations needed for gradient descent to converge.
- **Support Vector Machine (SVM): $O(N^2 \times D)$ to $O(N^3)$.**
 - SVM training complexity scales non-linearly with the number of samples.
 - While intensive on large datasets, the **LinearSVC** used in the code is optimized for high-dimensional sparse data like TF-IDF, allowing it to perform efficiently on the 10,000-sample subset.

7. Conclusion

In this assignment, both conventional and deep learning-based word vectorization techniques were explored and analyzed.

The results show that TF-IDF combined with Support Vector Machine provides the highest accuracy while maintaining reasonable computational efficiency. Bag of Words is faster but less effective, whereas Word2Vec with neural networks captures semantic relationships but requires more computational resources.

Overall, the choice of vectorization technique depends on the trade-off between accuracy and computational cost. For smaller datasets, conventional methods perform effectively, while deep learning methods are better suited for large-scale applications.

8. GitHub Repository

<https://github.com/kiranreddy89/Word-Vectorization-Techniques-in-NLP.git>

9. Sample Predictions

To demonstrate how the **TF-IDF + SVM** model handles different types of language, here are sample predictions based on the logic of your implementation and the specific results from your experiment. These examples highlight how the model interprets features and where it may encounter limitations.

Input Text	Predicted Sentiment	Behavior Explanation
"This movie was absolutely amazing and very interesting"	Positive	Words like "amazing" and "interesting" carry high TF-IDF weights, strongly signaling a positive classification.
"Worst movie I have ever watched, completely boring"	Negative	Dominant negative features like "worst" and "boring" lead the SVM to a negative prediction.

"The movie was okay, not great but not bad either"	Positive	The model identifies "okay" as slightly positive; in the absence of strong negative indicators, it leans toward a positive result.
"The movie was not bad"	Positive	Traditional count-based models often struggle with negations; here, the model likely leans positive because it treats words as independent features.